

INTERMEDIATE PROGRAMMER · FINAL PROJECT

Inventory & Order Management System

Aplikasi web multi-gudang dengan tiga peran, berlapis (Controller → Service → Repository), dan integritas stok yang terjaga saat proses berjalan bersamaan.

DISUSUN OLEH

Arief Aryudi Syidik

PROGRAM

PT Neuronworks Indonesia

SESI

Rabu, 7 Oktober 2026

Masalah dan pengguna

Tim gudang dan sales perlu mencatat pembelian dari supplier dan penjualan ke customer di beberapa gudang, dengan **angka stok yang bisa dipertanggungjawabkan**, termasuk ketika dua proses berjalan bersamaan.

Status penyelesaian

Semua fitur wajib selesai: master data, PO, SO dengan approval, goods receipt/issue, perpindahan stok, stock ledger, laporan CSV, dashboard, dan endpoint JSON.

Kontribusi saya: perancangan dan implementasi proyek. Sebagian besar kode, test, dan dokumentasi dibuat dengan bantuan Claude Code; saya bertanggung jawab memahami dan menjelaskannya (lihat slide terakhir).

AKTIVITAS	ADMIN	SALES	GUDANG
Buat dan ajukan SO	Ya	Miliknya	Tidak
Setujui / tolak SO	Ya*	Tidak	Tidak
Goods issue (SO)	Ya	Tidak	Ya
Goods receipt (PO)	Ya	Tidak	Ya
Kelola user	Ya	Tidak	Tidak

* Bukan untuk SO yang ia buat sendiri. Aturan ditegakkan di server, bukan hanya disembunyikan di UI.

Dua alur yang mewakili requirement

A · Purchase Order → penerimaan barang

- 1 Warehouse membuat PO (Draft) lalu **Order**
- 2 **Receive** sebagian → PartiallyReceived
- 3 Stok naik dan baris `stock_ledger` tipe Receipt, dalam **satu transaksi**

B · Sales Order → anti-oversell

- 1 Sales membuat SO; stok tampil lewat **Fetch API**; submit
- 2 Sales tidak bisa approve (server **403**); Admin approve
- 3 Warehouse **Fulfill** dengan qty melebihi stok → ditolak; stok dan ledger **tidak berubah**

Bukti perubahan data ditunjukkan langsung dari database (stok dan `stock_ledger`) sebelum dan sesudah.

Arsitektur berlapis dan alur request



Controller app/Controller · CrudController untuk 5 master data

Service app/Service · validasi, state machine, transaksi

Repository interface Dependency Inversion (ADR-001)

MySQL / In-Memory runtime sungguhan / test double

- Service hanya tahu **interface**, jadi diuji tanpa database.
- Tanpa framework dan ORM; PDO prepared statement di semua query.
- Error domain dipetakan terpusat di `Controller::handle` : 422 / 403 / 409 / 500, tanpa membocorkan detail.
- Diagram: `docs/architecture/class-diagram-as-built.md`

Integritas stok: tidak ada oversell

```
// SalesOrderService::fulfill()
$pdo->beginTransaction();
foreach ($items as $item) {
    $available = $stocks->lockForUpdate(...);
    // SELECT quantity ... FOR UPDATE
    if ($available < $item->qty) {
        throw InsufficientStockException;
    }
}
// decrement + tulis stock_ledger
$pdo->commit(); // gagal → rollBack()
```

- **Pessimistic row lock** InnoDB: dua goods issue pada produk dan gudang yang sama dipaksa antre.
- Cek stok dilakukan **setelah** lock, sehingga hasilnya tidak basi.
- Kekurangan di satu item **membatalkan seluruh transaksi**: tidak ada stok setengah berkurang.
- Terbukti lewat integration test terhadap MySQL asli dan test E2E.
- Keputusan dan alternatif: **ADR-002**

Frontend, JSON, dan Docker

Frontend

- HTML semantik, CSS sendiri, responsif
- Vanilla JS: validasi form, baris item dinamis
- Semua form dilindungi token CSRF (otomatis)

JSON API

- **Fetch API:** GET `/api/products/{sku}/availability`
- 401 tanpa login, 404 SKU tidak ada
- Error domain jadi JSON 422 / 403 / 409 / 500

Docker

- `docker compose up --build` : app + MySQL
- Schema dan seed diimpor otomatis
- Multi-stage, **non-root**, tanpa tool dev, healthcheck
- MySQL hanya di **127.0.0.1**; diverifikasi dari clone bersih

Keamanan: ancaman, kontrol, bukti

ANCAMAN	KONTROL	BUKTI
CSRF	Token per-sesi di semua form POST, divalidasi di front controller	SecurityE2ETest
Sesi dibajak / basi	HttpOnly, SameSite, timeout; user divalidasi ulang tiap request	Sesi, nonaktif, role
Brute force login	Throttling DB (5 per akun, 20 per IP); respons waktu-konstan	Lockout E2E
Akses objek (IDOR)	Sales hanya SO miliknya; tidak bisa setuju SO sendiri	Temuan diperbaiki
Injeksi	Prepared statement; output di-escape + CSP; CSV dinetralkan	Audit + test
Upload berbahaya	mime + <code>getimagesize</code> , 2MB, nama acak, PHP mati di uploads	Diuji di Apache
Container	Non-root, tanpa tool dev, header keamanan, MySQL localhost	Dicek di container

Sisa risiko (jujur): TLS dari reverse proxy, rate limiting API, MFA. Rincian: `docs/quality/security-review.md` , ADR-004. SonarQube: 0 isu, 0 hotspot, security A.

Test dan SonarQube



METRIK	SCAN AWAL	AKHIR
Isu terbuka	192	0
Reliability / Security / Maintainability	C / A / A	A / A / A
Coverage	21,1%	100%
Duplikasi	8,1%	0%

- E2E menembak aplikasi lewat HTTP dan merekam coverage dari server (ADR-003), digabung dengan Unit + Integration.
- PHPStan level 5 dan PHPCS PSR-12: 0 error.
- Dua pengecualian beralasan: `php:S2003` (view/config) dan `php:S2092` (flag Secure cookie aktif otomatis di HTTPS).

Perbaikan utama yang dilakukan

TEMUAN	TEKNIK	HASIL
5 controller CRUD hampir identik	Template Method (CrudController)	≈200 baris duplikat hilang
Query builder PO/SO/ledger berulang	Extract Superclass (AbstractMySqlRepository)	SQL dibandingkan: identik
HTML form/daftar disalin-tempel	Partial view (views/partials/)	Render dibandingkan: identik
43 parameter \$request tidak terpakai	Router memetakan argumen by nama (reflection)	Handler hanya deklarasi yang dipakai
Method besar (kompleksitas, banyak return)	Extract Method	Di bawah ambang SonarQube
Form tanpa label terasosiasi	Label membungkus kontrol, aria-label	Isu aksesibilitas 0

Dokumentasi: class diagram initial vs as-built, ADR-001/002/003, refactor-log.md , sonarqube-report.md .

Kendala, keterbatasan, dan langkah berikutnya

Kendala dan solusi

- Stok benar saat konkuren → transaksi + row lock.
- Clone bersih gagal jalan (bind mount menimpa `vendor/`) → ketahuan saat verifikasi, diperbaiki.
- Coverage Controller/view awalnya 0% → test E2E lewat HTTP.

Keterbatasan (jujur)

- TLS perlu reverse proxy; rate limiting baru untuk login.
- API memakai session cookie; `ApiController` melewati Service.
- 100% adalah *line* coverage, bukan semua kombinasi input.
- Tanpa CI/CD.

Prioritas berikutnya

1 Rate limiting API · 2 TLS + reverse proxy · 3 Audit log master data · 4 MFA & token API

Penggunaan AI

- Sebagian besar kode, test, dan dokumentasi dibuat dengan **Claude Code** (Anthropic).
- Subagent paralel dipakai untuk refactor duplikasi di worktree terpisah, lalu perilakunya **dibandingkan sebelum dan sesudah** dan diuji ulang.
- Saya memverifikasi hasil: build Docker dari clone bersih, test, SonarQube, dan smoke test semua route per role.

Tanggung jawab

Saya bertanggung jawab memahami dan menjelaskan seluruh kode, termasuk

`SalesOrderService::fulfill` , `Router` , dan `CrudController` .

Rincian: `ai-usage-log.md`

TERIMA KASIH

Tanya jawab

Repository, README, ADR, class diagram, laporan SonarQube, dan laporan coverage siap ditelusuri langsung dari kode.

AKUN DEMO

admin@ioms.test · Password123!

APLIKASI

localhost:8080

SONARQUBE

localhost:9001